MATLAB® Workshop

Joshua Oliva Parra

IEEE PES - UdeC

Universidad de Concepción

2026 - 1

¿Qué es MATLAB®?

Plataforma de programación diseñada espeíficamente para ingenieros y científicos

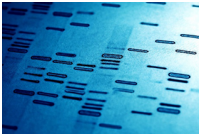
Características

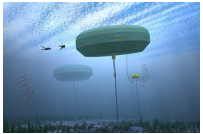
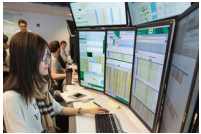
- Lenguaje basado en Matrices.
- Provee una sintaxis matemática mas natural.

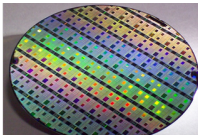
¿Qué se puede hacer en MATLAB®?

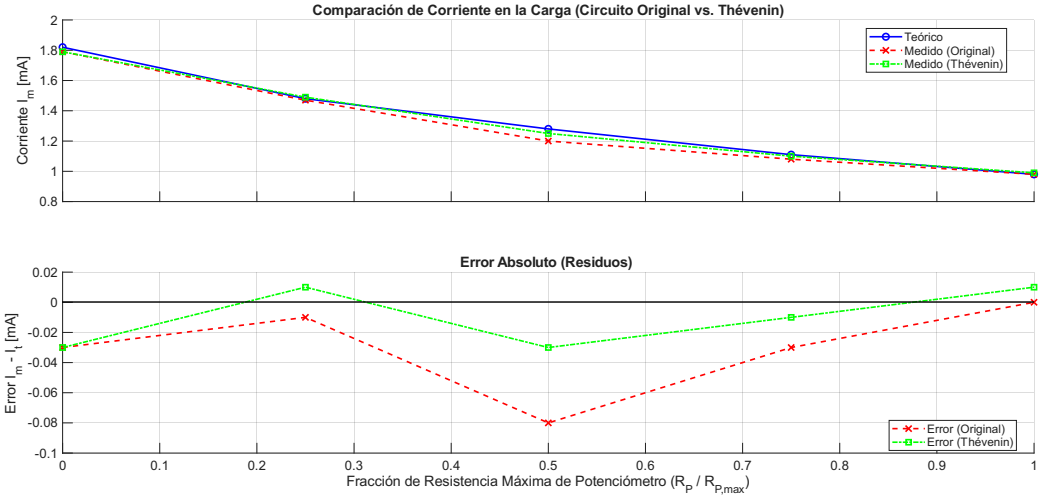
- Analizar datos.
- Desarrollar algoritmos.
- Crear modelos y aplicaciones.

Industrias que usan MATLAB®






Comparación de Corriente en la Carga (Circuito Original vs. Thévenin)
Corriente I_m [mA]
2
1.8
1.6
1.4
1.2
1
0.8
Teórico
Medido (Original)
Medido (Thévenin)
Error Absoluto (Residuos)
Error I_m - I_t [mA]
0.02
0
-0.02
-0.04
-0.06
-0.08
-0.1
Error (Original)
Error (Thévenin)
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
Fracción de Resistencia Máxima de Potenciómetro (R_P / $R_{P,max}$)

¿Qué aprenderemos hoy?

Objetivo General

- Al finalizar el taller, podrán desarrollar scripts para procesar, analizar y visualizar datos, integrando operaciones vectoriales.
- Si queda tiempo también se presentarán las bases de las estructuras lógicas de programación más comunes.

Objetivos Específicos

- Conocer los conceptos básicos de programación al igual que el uso básico del entorno de MATLAB®.
- Saber donde buscar la documentación de las distintas funciones integradas.
- Aprender la sintaxis de las estructuras lógicas básicas (**if**, **for**).

Bloque 1: Fundamentos de Arreglos

- Generar arreglos (Vectores y matrices) de distintas formas.
- Realizar operaciones aritméticas y lógicas usando arreglos.
- Extraer elementos específicos de estos mediante técnicas de indexación, máscaras y funciones útiles.

Bloque 2: Extracción y Visualización

- Importar datos desde archivos externos y desempaquetarlos.
- Crear gráficos para presentar datos.
- Dar formato visual a los gráficos.

Entorno de MATLAB®

Las 4 Ventanas Principales

MATLAB® se divide en secciones que facilitan el flujo de trabajo:

1. **Current Folder:** La carpeta donde estás trabajando.
2. **Command Window:** Escribes una orden y se ejecuta al dar *Enter*.
3. **Workspace:** El almacén de memoria. Aquí verás todas las variables que has creado o que esté activas.
4. **Editor / Live Editor:** Donde escribes tus programas (scripts) para guardarlos y correrlos después.

Entorno de MATLAB®

Scripts

Un **Script** dicta una lista de instrucciones directamente a la Ventana de Comandos en orden. Para mantener el control, usamos:

- **El Silenciador (`;`):** Se ubica al final de una línea para que MATLAB® trabaje en **silencio**.
- **Comentarios (`%`):** Estos permiten dejar notas o texto que MATLAB® no ejecuta.

Entorno de MATLAB®

Comandos Útiles

Es buena práctica iniciar programas con estos comandos:

Comandos de Mantenimiento

- `clc`: Borra el texto de la Ventana de Comandos.
- `clear`: Borra todas las variables de la memoria (Workspace).

Consejo: Usa `clear; clc;` *al inicio de tu script para evitar errores de pruebas anteriores.*

Bloque 1
Fundamentos de Arreglos

¿Qué es un arreglo?

Un arreglo es la estructura de datos **fundamental** de MATLAB®. Se define como una colección **n-dimensional** de componentes homogéneos (numeros reales, complejos, etc.).

Características

- Dimensión (N).
- Tamaño.
- Clase (`codigo`, `codigo`, `codigo`, etc.).

En MATLAB® TODO es un arrelo.

Creación de Arreglos

Creación Manual

Para crear un arreglo manualmente, se utilizan **corchetes** `[]` como contenedor y delimitadores internos para definir la estructura:

Delimitadores de Dirección

- **Coma `,` (o espacio):** Separa elementos en la misma fila. El arreglo crece hacia la derecha (horizontal).
- **Punto y coma (`;`):** Separa filas. El arreglo crece hacia abajo (vertical).

Nota: Para crear una matriz (arreglo con mas de una fila), ***todas las filas deben tener el mismo número de elementos.***

Creación de Arreglos

Generador de Secuencias

Operador de **Dos Puntos** (`:`)

Se usa cuando conoces el **paso o incremento** entre los números.

- **Sintaxis:** `inicio : paso : fin`
- **Ejemplo:** `t = 0:2:10`. Genera: `[0, 2, 4, 6, 8, 10]`

Nota: Si se omite el paso (`0:5`), MATLAB® asumirá que el paso es 1.

Función `linspace`

Se usa cuando conoces la cantidad total de punto que quieres, sin importar el paso.

- **Sintaxis:** `linspace(inicio, fin, num_puntos)`
- **Ejemplo:** `linspace(0, 10, 5)`. Genera: `[0, 2.5, 5, 7.5, 10]`.

Creación de Arreglos

Arreglos con valores predefinidos

Existen funciones que generan arreglos con valores ya inicializados.

Funciones

- **Función** `zeros`: Genera un arreglo donde todos los elementos son 0.
  Sintaxis: `zeros(filas, columnas)`.
- **Función** `ones`: Genera un arreglo donde todos los elementos son 1.
  Sintaxis: `ones(filas, columnas)`.
- **Función** `rand`: Genera un arreglo donde todos los elementos son valores aleatorios entre 0 y 1.
  Sintaxis: `rand(filas, columnas)`.

Operaciones Básicas con Arreglos

Operaciones Vectoriales

Las operaciones vectoriales comprenden las asociadas al álgebra lineal (Se siguen las mismas reglas de dimensiones para sumas y productos).

- Sumar/Restar 2 vectores/matrices: `Mat1 + Mat2`.
- Multicación/División por escalar: `Mat1 * n`.
- Transposición de vectores o matrices (Operador `´`): `Vec´`.
- Producto Interno entre 2 vectores: `dot(Vec1, Vec2)`.
- Multiplicación Matricial: `Mat1 * Mat2`
- Potencia Matricial: `Mat1^n`

Operaciones Básicas con Arreglos

Operaciones Elemento a elementos

Existen operaciones aparte de las que dicta el álgebra lineal, útiles para trabajar con magnitudes grandes de datos.

- Suma/Resta entre escalares y arreglos: `Arr + n`
- Multiplicación/División elemento a elemento: `Arr .* n`
- Potencia elemento a elemento: `Arr .^ n`

Indexación (Indexing)

La **indexación** es el proceso de acceder a uno o varios elementos específicos dentro de un arreglo utilizando su posición o "dirección" numérica.

Reglas de Oro en MATLAB®

- **Base-1:** El primer elemento siempre tiene el índice 1.
- Se utilizan **paréntesis curvos** `()` para indicar la posición.
- **Sintaxis:** `A(índice)`

Notas

- El uso de `end` en un índice referencia siempre al **último** elemento de la dimensión.
- Para arreglos de múltiples dimensiones, se usa la coma (`,`) para separar dimensiones.

Indexación (Indexing)

Operador `:`

El operador `:` permite extraer múltiples elementos a la vez, creando un 'recorte' del arreglo original.

Sintaxis: `Vec(inicio:fin)` o `Vec(inicio:paso:fin)`

Dado `Vec = [12, 13, 14, 15, 16]`

- `Vec(2:4)` $\Longrightarrow$ `[13, 14, 15]`
- `Vec(end:-1:4)` $\Longrightarrow$ `[16, 15]` *(Paso negativo)*

Nota:

- El operador `:` por sí solo, selecciona **todos** los elementos de esa dimensión (ej: `Vec(:)`).

Máscaras Lógicas

Una máscara es un arreglo de **booleanos** (0 y 1) que resulta de hacerle una pregunta lógica a nuestros datos.

Operadores de comparación:

- `==` (Igual), `<`, `>` (Existen mas).

Operadores Lógicos

- `&` (Conjunción), `|` (Disyunción), `~` (Negación).

¿Qué genera MATLAB®?

```
Temp = [25, 999, 28];
Temp == 999;
```

$\Longrightarrow$ `[0, 1, 0]` *(Falso, Verdadero, Falso)*

Nota: El resultado es un "mapa"que indica dónde se cumple la condición.

Máscaras Lógicas

Aplicación

Al usar una máscara como **índice**, MATLAB® actúa solo sobre las posiciones marcadas con 1 (True).

- **Extracción (Filtrar):** Crea un nuevo vector solo con los datos que nos interesan.
  `buenos = Temp(Temp < 100);`
- **Asignación (Reemplazar):** Modifica valores específicos en el vector original.
  `Temp(Temp == 999) = NaN;`

Consejo

Asignar `NaN` (Not a Number) es la forma estándar de "borrar" un error sin eliminar la posición en el vector.

Funciones Útiles

MATLAB® ofrece funciones integradas para extraer información estadística y estructural de los arreglos de forma inmediata.

Funciones

- `size(A)`: Devuelve el número de filas y columnas $[m, n]$.
- `sum(A)`: Calcula la suma de todos los elementos.
- `mean(A)`: Calcula el promedio aritmético.
- `max(A)` / `min(A)`: Encuentra el valor máximo o mínimo.

*Nota: Por defecto, en matrices estas funciones operan por **columnas**.*

Bloque 2
Extracción y Visualización de Datos

Tablas

Las **tablas** son un **contenedor** que guarda varios vectores juntos en forma de columnas.

Características

- En un **Arreglo o Matriz:** Todos los datos tienen que ser del mismo tipo.
- En una **Tabla:** Cada columna es un "paquete" independiente con nombre propio (etiqueta).
- Para sacar los datos de una columna, no es necesario saber la posición de esta, solo su nombre.

Dados dos vectores se puede crear una tabla mediante la función `table(vec1, vec2)`.
El argumento `'VariableNames'` permite nombrar las columnas.

Consideración

Los vectores deben tener el mismo largo y deben ser vectores columna.

Tablas

Desempaquetado

El **desempaquetado** consiste en extraer una o varias columnas de una tabla para trabajar con ellas como vectores independientes.

El operador punto `.`

- Permite acceder a una columna directamente por su nombre.
- **Sintaxis:** `vector = MiTabla.NombreVariable;`

¿Por qué desempaquetar?

- Para realizar cálculos matemáticos rápidos sobre el vector.
- Para pasar los datos a funciones de graficado como `plot(t, temp)`.

Estructuras de Datos

Importación con `readtable`

Para trabajar con datos reales (como un archivo de Excel o CSV), el primer paso es traerlos a MATLAB®.

¿Cómo se hace?

Inicialmente se necesita la ruta donde se encuentren los datos, posteriormente se utiliza la función: `tabla = readtable('ruta/archivo.csv');`

- **Facilidad:** MATLAB® lee automáticamente los nombres de las columnas y organiza todo por ti.

Visualización de Datos

Gráficos Básicos

La función principal para graficar en 2D es la función `plot`.
La sintaxis primordial es:

- `plot(y)`: Grafica los datos contra su propio índice $(1, 2, 3...)$.
- `plot(x, y)`: Grafica `y` en función de `x`.

Importante

Para que `plot(x, y)` funcione, ambos vectores deben tener **la misma longitud**.

Visualización de Datos

Control Técnico

Una vez generada la gráfica, usamos estos comandos para controlar qué vemos y cómo se comporta la ventana:

- `hold on / hold off`: Permite dibujar múltiples líneas en la misma gráfica sin borrar la anterior.
- `xlim([min, max])`: Ajusta los límites del eje horizontal.
- `ylim([min, max])`: Ajusta los límites del eje vertical.

Visualización de Datos

Estética y Formato

Para que un gráfico sea informativo, debemos añadir contexto y estilo:

- **Estilo de línea (Argumentos de la función `plot`):**
  - `'LineWidth'`: Grosor de la línea (ej: 2).
  - `'Color'`: Color personalizado (ej: `'r'` para rojo).
- **Etiquetas:** `title('Título')`, `xlabel('Eje X')`, `ylabel('Eje Y')`.
- **Rejilla:** `grid on` ayuda a la lectura visual de los puntos.

Código de Ejemplo

```
plot(t, voltaje, 'Color', 'b', 'LineWidth', 1.5);
grid on; title('Voltaje v/s Tiempo'); xlabel('Segundos');
```

Mini Proyecto de Aplicación